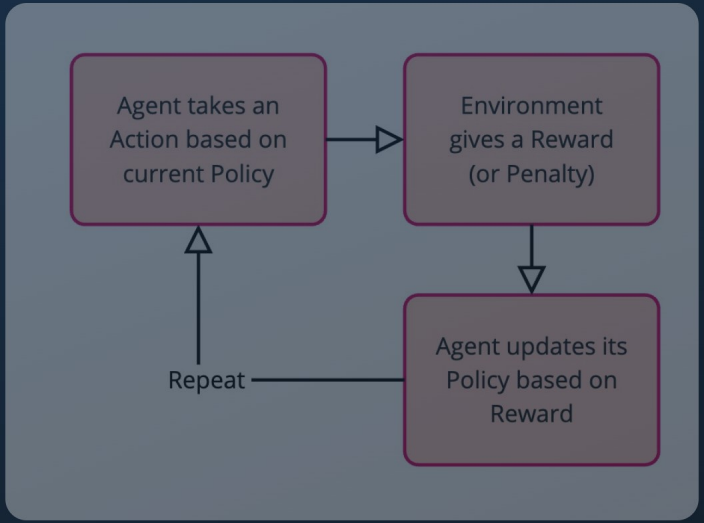
DOĞAL DIL İŞLEME DERSI

Hugging Face LLM Course · Bölüm 12

Ünite 12: Akıl Yürüten Modeller İnşa Etmek

Pekiştirmeli öğrenme, DeepSeek-R1 ve GRPO



Gökberk Eren Günay
Emir Kalyoncu
Sultan Ateş
İlker İzci

SUNUMUN AKIŞI

Dört bölüm, tek hikâye: bir modele düşünmeyi öğretmek

1

Pekiştirmeli öğrenmeye giriş

Büyük resim, terimler, Open R1, RL'nin parçaları, RLHF, PPO / DPO / GRPO

Sayfa 12.1 · 12.2

2

DeepSeek-R1 makalesi

Aha anı, 4 eğitim aşaması, GRPO'ya ilk bakış, sözde kod, sonuçlar

Sayfa 12.3

3

GRPO ileri düzey ve TRL

Avantaj, hedef fonksiyon, clip, KL, kodlar ve ödül fonksiyonları

Sayfa 12.3b · 12.4

4

Uygulama ve deney

TRL ve Unsloth ile GRPO, kendi deneyimiz, değerlendirme soruları

Sayfa 12.5 · 12.6

BÖLÜM 1 · SAYFA 12.1 VE 12.2

Pekiştirmeli öğrenmeye giriş

Open R1 nedir, RL nasıl çalışır, dil modellerinde neden gerekli?

1

GIRIŞ · ÜNİTENİN BÜYÜK RESMİ

Tek bir soru: bir dil modeline düşünmeyi nasıl öğretiriz?

1 · SORUN

LLM'ler akıcı yazar ama çok adımlı akıl yürütme isteyen matematik ve bulmacalarda zorlanır.

12.1

2 · FİKİR

Pekiştirmeli öğrenme: model cevap üretir, iyi cevap ödül alır, model ödüle göre gelişir.

12.2

3 · KANIT

DeepSeek-R1: büyük ölçekte RL ile model akıl yürütmeyi kendi kendine geliştirdi.

12.3

4 · YÖNTEM

GRPO: bir soruya birden çok cevap üret, cevapları kendi aralarında karşılaştır.

12.3b · 12.4

5 · UYGULAMA

TRL ve Unsloth ile GRPO'yu birkaç satır kodla çalıştır, sonucu ölç.

12.5 · 12.6

SUNUM BOYUNCA Hep aynı döngüye döneceğiz: **cevap üret → puanla → karşılaştır → modeli güncelle**.

GIRIŞ · KÜÇÜK SÖZLÜK

Sunum boyunca geçecek sekiz terim

LLM
Büyük dil modeli. Bir sonraki kelimeyi tahmin ederek metin üretir.

Token
Modelin okuduğu en küçük birim: kelime, kelime parçası ya da noktalama.

İnce ayar
Hazır bir modeli belirli bir amaç için ek eğitimle geliştirmek.

Pekiştirmeli öğrenme
Deneme yanılmayla, ödüle bakarak öğrenme (RL).

Politika
Modelin hangi durumda ne yapacağını seçme stratejisi. Eğitimde değişen budur.

Ödül fonksiyonu
Bir cevaba puan veren fonksiyon. Örneğin doğruysa 1, yanlışsa 0.

Avantaj
Bir cevabın, aynı gruptaki diğer cevaplara göre ne kadar iyi olduğu.

KL ıraksaması
Eğitilen modelin başlangıçtaki modelden ne kadar uzaklaştığının ölçüsü.

İlk beşini bu bölümde, son üçünü Bölüm 2 ve 3'te ayrıntılı göreceğiz.

Model önce düşünür, sonra cevap verir

LLM'ler birçok üretken görevde başarılı, ama **birden fazla akıl yürütme adımı** isteyen bulmaca ve matematik problemlerinde zorlanıyor.

Open R1, pekiştirmeli öğrenme ile LLM'leri "düşünmeye" teşvik eden açık kaynaklı bir topluluk projesi.

Model hem **düşünce** hem **cevap** üretir; ikisi ayrı ayrı okunabilecek şekilde yapılandırılır.

Problem: 3 elmam ve 2 portakalım var. Toplamda kaç meyvem var?

Düşünce: Elma ve portakal sayılarını toplamam gerekiyor.

Cevap: 5

```
<think>I need to add the number of apples and
oranges to get the total number of pieces of
fruit.</think>
5
```

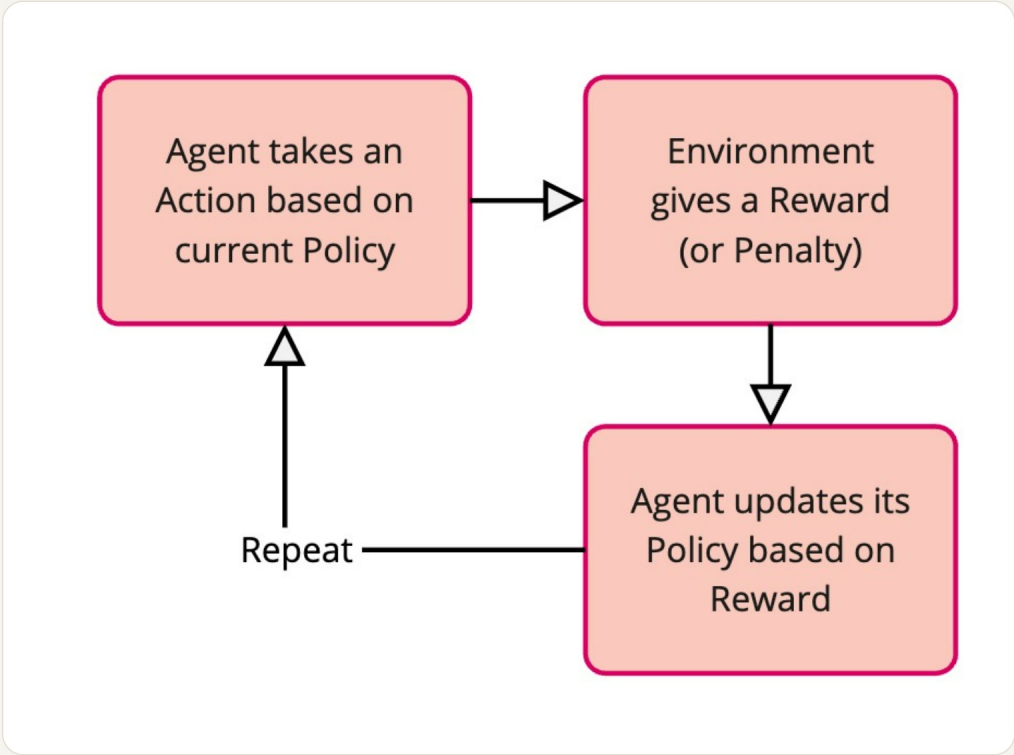
Modelin beklenen çıktı formatı: düşünce <think> etiketinin içinde, cevap dışında.

Köpeğe "otur" öğretmek gibi: doğru davranışa ödül

Köpek otururs a mama ve övgü alır. Zamanla oturmayı ödülle ilişkilendirir ve "Otur!" dendiğinde oturma olasılığı artar.

Bu geri bildirime **ödül (reward)** denir.

> Köpeğin yerinde **dil modeli** (ajan), eğitmenin yerinde geri bildirim veren **ortam** var.



Ajan eylem yapar → ortam ödül/ceza verir → ajan politikasını günceller → tekrar

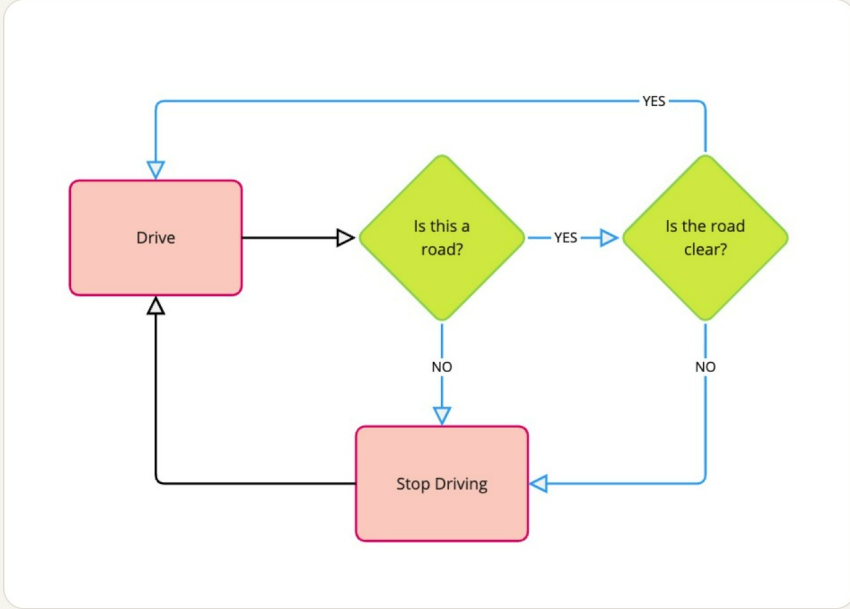
Beş kavram: ajan, ortam, eylem, ödül, politika

Kavram	Köpek örneğinde	Dil modelinde (LLM)
Ajan (Agent)	Köpek	Eğitmek istediğimiz LLM'nin kendisi; öğrenen ve karar veren taraf
Ortam (Environment)	Ev ve eğitmen	Kullanıcılar ya da kurduğumuz simüle senaryo; geri bildirim verir
Eylem (Action)	Otur, kalk, havla	Kelime üretmek, bir cevap seçmek, nasıl yanıt vereceğine karar vermek
Ödül (Reward)	Mama ve övgü	Cevabın ne kadar yardımcı, doğru veya zararsız olduğunu gösteren sayı
Politika (Policy)	"Otur" denince ne yapacağını bilmesi	Hangi durumda hangi eylemin seçileceği stratejisi; RL'de asıl öğrenilen şey

AKILDA KALSIN Ajan bir **eylem** yapar, ortam **ödül** verir; ajan zamanla daha çok ödül getiren **politikayı** öğrenir.

Deneme ve yanılma ile öğrenme

1. **Gözlem:** ajan ortamı ve mevcut durumu gözlemler
2. **Eylem:** politikasına göre ne yapacağına karar verir
3. **Geri bildirim:** ortam bir ödül verir
4. **Öğrenme:** yüksek ödül getireni pekiştirir, düşükten kaçınır
5. **Yineleme:** döngü tekrar eder, karar verme gelişir



Sür → Bu bir yol mu? → Yol açık mı? Cevap "hayır" ise: sürmeyi durdur.

Akıcı olmak yetmez: model yardımcı ve güvenli de olmalı

Ön eğitim
Sonraki kelimeyi tahmin eder; akıcı, dil bilgisi doğru metin

→

Denetimli ince ayar
Yapılandırılmış çıktı; ama yanlış veya önyargılı olabilir

→

Pekiştirmeli öğrenme
İstenen nitelikleri kazandırmak için ince ayar

Yardımcı
Faydalı ve ilgili bilgi sağlar

Zararsız
Toksik, önyargılı veya zararlı içerikten kaçınır

İnsan tercihleriyle hizalı
İnsanların doğal ve yardımcı bulduğu şekilde yanıt verir

AKILDA KALSIN Ön eğitim modele **dili** öğretir; RL ise modele **nasıl davranacağını** öğretir.

RLHF: ödülün yerine insan tercihi

1. İnsan tercihlerini topla

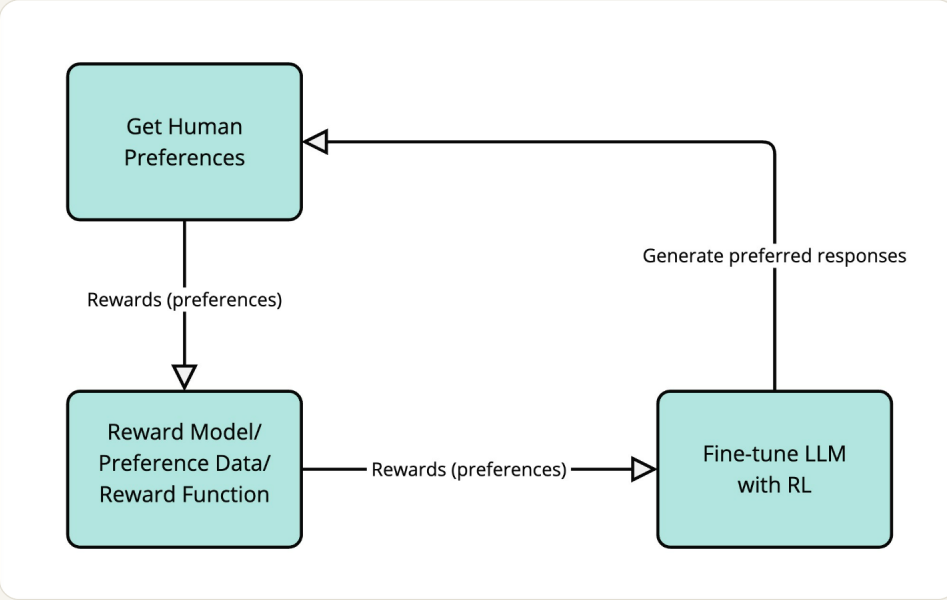
Aynı soruya verilen iki cevaptan hangisi daha iyi? Örn. "Fransa'nın başkenti neresidir?"

2. Ödül modeli eğit

Ayrı bir model, insanların hangi cevabı tercih edeceğini tahmin etmeyi öğrenir

3. LLM'ye RL ile ince ayar yap

Ödül modeli ortam olur: LLM cevap üretir, ödül modeli puanlar



GPT-4, Gemini ve DeepSeek-R1 gibi modellerin eğitiminde kullanıldı.

12.2 · NEDEN GRPO?

Üç yöntem, üç farklı ödül kaynağı

PPO

Proximal Policy Optimization

RLHF için ilk yüksek etkili tekniklerden. **Ayrı bir ödül modelinden** gelen ödülle politika gradyanı uygular.

Uzunluk fonksiyonu: kısa cevabı ödüllendir

DPO

Direct Preference Optimization

Ödül modelini kaldırır. **Seçilen ve reddedilen** cevapları bir sınıflandırma görevi gibi kullanır.

Matematik çözücü: doğru çözümü ödüllendir

GRPO

Group Relative Policy Optimization

Benzer örnekleri **grup hâlinde karşılaştırır**. Ödül bir modelden de, **herhangi bir fonksiyondan** da gelebilir.

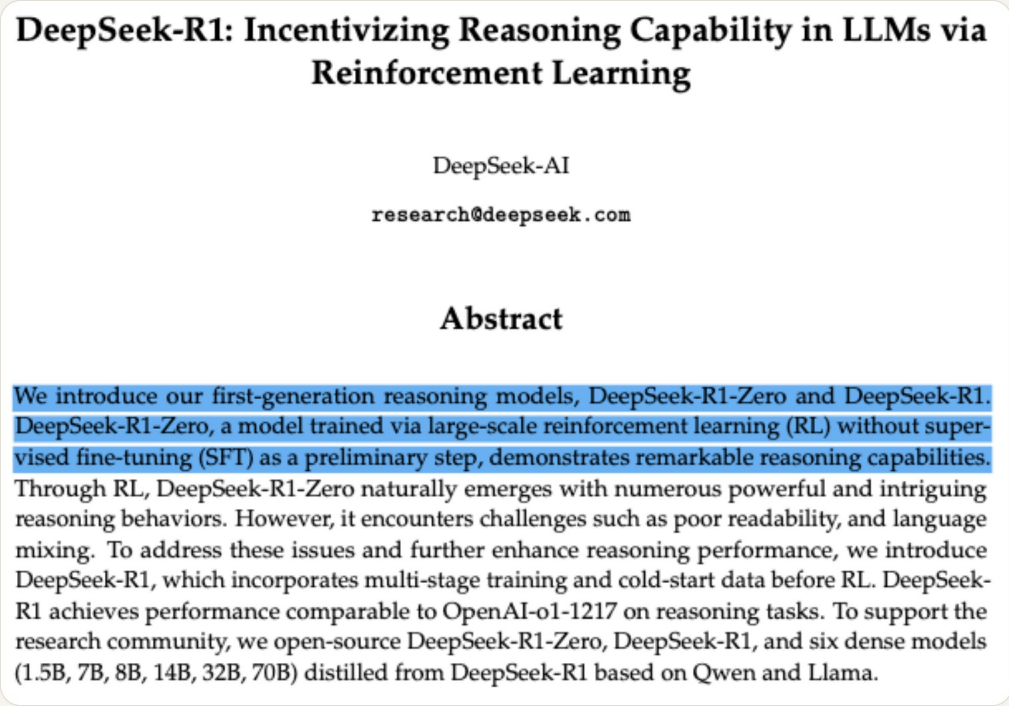
Olgusal doğruluk: doğru bilgiyi ödüllendir

2

DeepSeek-R1 makalesini anlamak

Aha anı, dört eğitim aşaması, GRPO'ya ilk bakış ve sonuçlar

Soru: Saf RL, SFT olmadan akıl yürütmeyi öğretebilir mi?



DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

Abstract

We introduce our first-generation reasoning models, DeepSeek-R1-Zero and DeepSeek-R1. DeepSeek-R1-Zero, a model trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step, demonstrates remarkable reasoning capabilities. Through RL, DeepSeek-R1-Zero naturally emerges with numerous powerful and intriguing reasoning behaviors. However, it encounters challenges such as poor readability, and language mixing. To address these issues and further enhance reasoning performance, we introduce DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. To support the research community, we open-source DeepSeek-R1-Zero, DeepSeek-R1, and six dense models (1.5B, 7B, 8B, 14B, 32B, 70B) distilled from DeepSeek-R1 based on Qwen and Llama.

DeepSeek-R1: Pekiştirmeli öğrenme yoluyla LLM'lerde akıl yürütmeyi teşvik etmek

O zamana kadar popüler LLM'lerin hepsi bir miktar **denetimli ince ayar (SFT)** gerektiriyordu.

Makale, **GRPO** adlı pekiştirmeli öğrenme algoritmasını kullanıyor.

Özetten: R1, akıl yürütme görevlerinde **OpenAI-o1-1217** ile karşılaştırılabilir performansa ulaşıyor.

12.3 · ÇIĞIR AÇAN "AHA" ANI

Model, kimse öğretmeden kendi hatasını düzeltmeyi öğrendi

1 · İlk deneme
"Bu parça renge göre buraya gelmeli"

→

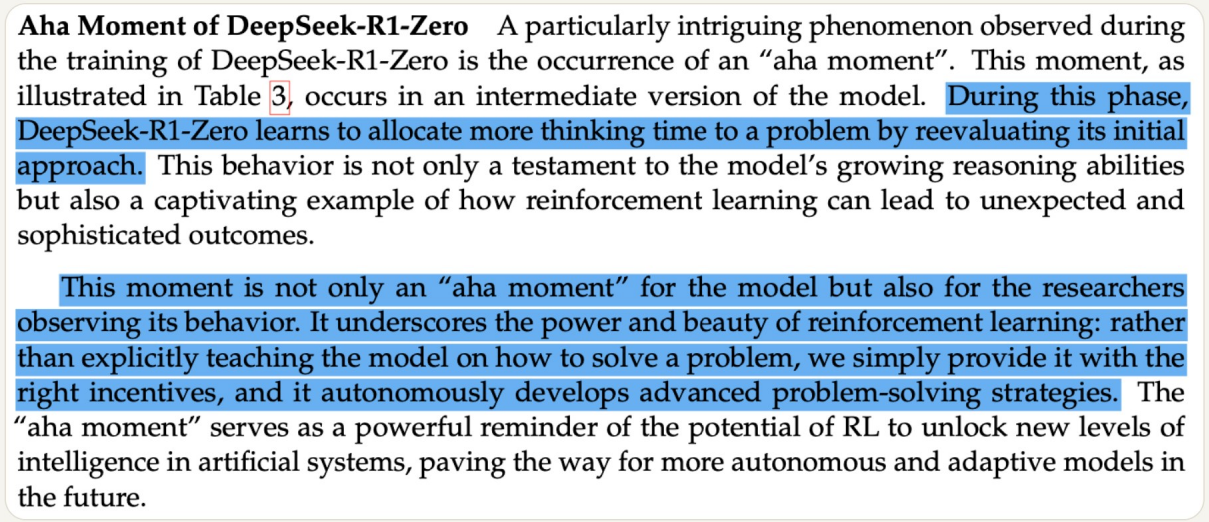
2 · Fark etme
"Ama durun, şekli tam oturmuyor"

→

3 · Düzeltme
"Ah, aslında şuraya ait"

→

4 · Açıklama
"Çünkü burada hem renk hem şekil uyuyor"



Aha Moment of DeepSeek-R1-Zero A particularly intriguing phenomenon observed during the training of DeepSeek-R1-Zero is the occurrence of an "aha moment". This moment, as illustrated in Table 3, occurs in an intermediate version of the model. During this phase, DeepSeek-R1-Zero learns to allocate more thinking time to a problem by reevaluating its initial approach. This behavior is not only a testament to the model's growing reasoning abilities but also a captivating example of how reinforcement learning can lead to unexpected and sophisticated outcomes.

This moment is not only an "aha moment" for the model but also for the researchers observing its behavior. It underscores the power and beauty of reinforcement learning: rather than explicitly teaching the model on how to solve a problem, we simply provide it with the right incentives, and it autonomously develops advanced problem-solving strategies. The "aha moment" serves as a powerful reminder of the potential of RL to unlock new levels of intelligence in artificial systems, paving the way for more autonomous and adaptive models in the future.

Makaleden: "Modele bir problemi nasıl çözeceğini açıkça öğretmek yerine ona sadece **doğru teşvikleri** veririz; o da kendi kendine problem çözme stratejileri geliştirir."

DeepSeek-R1-Zero ve DeepSeek-R1

Özellik	R1-Zero	R1
Eğitim yaklaşımı	Saf RL	Çok aşamalı (SFT + RL)
İnce ayar	Yok	Denetimli ince ayar
Akıl yürütme	Kendiliğinden ortaya çıkan	Geliştirilmiş
Temel özellik	Güçlü akıl yürütme, okunabilirlik sorunlu	Daha iyi dil tutarlılığı ve okunabilirlik

AIME performansı

%71,0

R1-Zero

%79,8

R1

R1'in dört eğitim aşaması

1

Soğuk başlangıç

Kalite temeli

V3-Base, R1-Zero'dan alınan binlerce doğrulanmış, kaliteli örnekle ince ayarlanır.

2

Akıl yürütme RL

Yetenek inşası

Matematik, kod, bilim, mantık. Kural tabanlı ödül; görevler doğrulanabilir, ayrı ödül modeli yok.

3

Ret örneklemesi

Kalite kontrolü

Model örnek üretir, DeepSeek-V3 kalite hakemi olarak süzer, kalanlarla SFT yapılır.

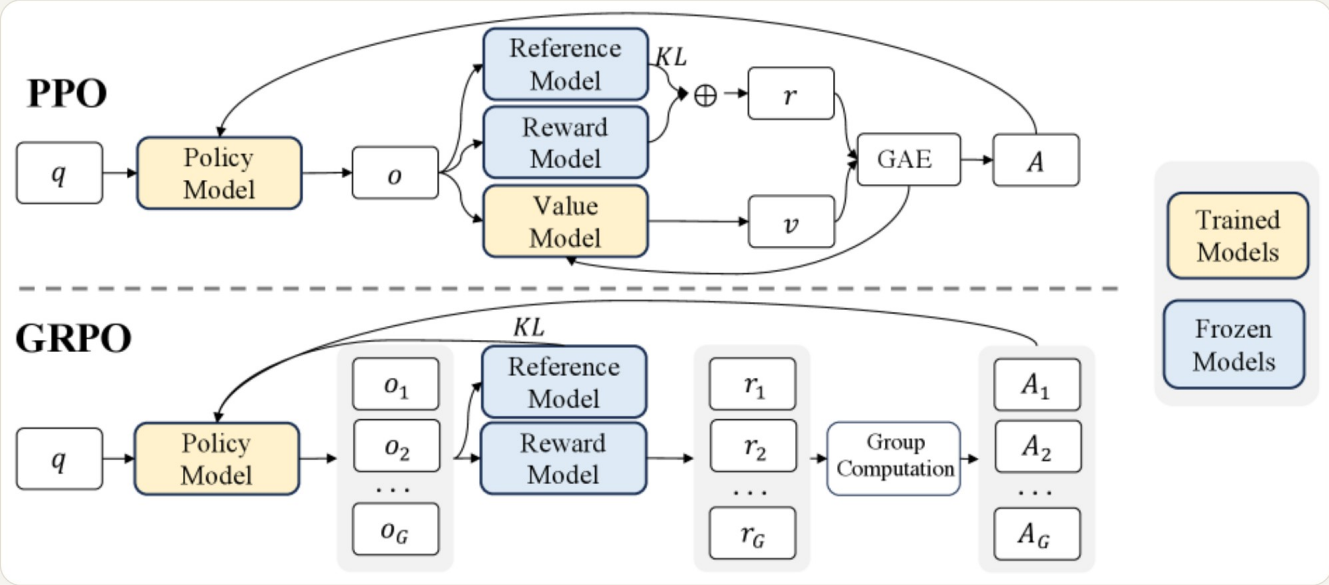
4

Çeşitli RL

Geniş hizalama

Hibrit ödül: kesin görevlerde kural, öznel görevlerde LLM geri bildirimi.

PPO'daki değer modeli yok, taban çizgisi grup puanlarından



DeepSeekMath, Şekil 4. Sarı: eğitilen modeller · Mavi: dondurulmuş (eğitilmeyen) modeller

PPO

Tek çıktı o, ayrı bir değer modeli v ile avantaj A

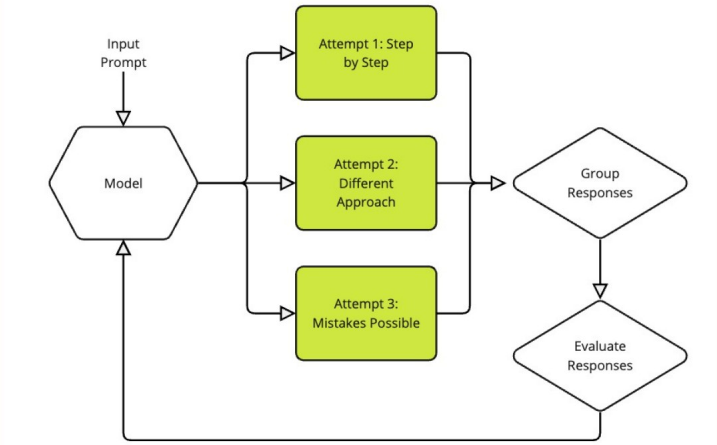
GRPO

G çıktı o_1...o_G, ödüller r_1...r_G, grup hesaplamasıyla A_1...A_G

Değer (eleştirmen) modeli çoğu zaman politika modeliyle aynı boyutta; onu kaldırmak eğitim maliyetini düşürür.

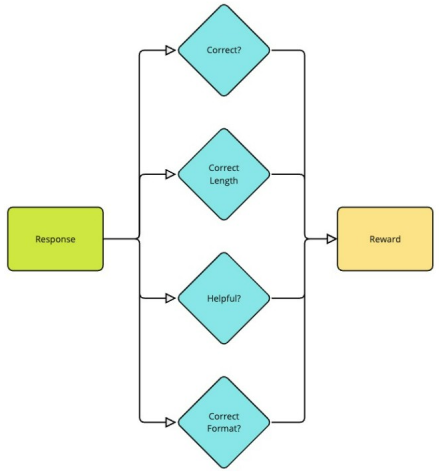
Birden fazla çözüm üret, sonra onları puanla

1. Grup oluşturma



Aynı soru için 4, 8 veya 16 deneme. Birden fazla öğrencinin çözümünü yan yana koymak gibi.

2. Tercih öğrenme



Doğru mu? Uzunluk uygun mu? Yardımcı mı? Format doğru mu? Puanlar ödüle dönüşür.

Mutlak puan değil, gruba göre puan: "çan eğrisine göre not"

```
Advantage = (reward - mean(group_rewards)) / std(group_rewards)
```

Avantaj = (ödül – grup ödüllerinin ortalaması) / grup ödüllerinin standart sapması

3. Optimizasyon: iki prensip

Başarılı çözümlere benzeyenleri teşvik et, daha az etkili yaklaşımlardan uzaklaş.

KL ıraksama cezası ile modelin bir anda sert değişmesini engelle.

Neden daha kararlı?

İki çözüm yerine birden fazla çözüme birlikte bakar.

Grup içi normalleştirme ödül ölçeği sorunlarını önler. KL cezası bir güvenlik ağıdır.

Algoritmanın tamamı tek ekranda

```
Algorithm GRPO:
1. For each training iteration:
  a. Set reference_policy = initial_policy
  b. For each prompt in batch:
    i. Generate group_size different outputs
    ii. Compute rewards using reward_function
    iii. Normalize rewards within group:
      adv = (reward - mean(rewards)) / std(rewards)
    iv. Update policy by maximizing:
      min(prob_ratio * adv,
        clip(prob_ratio, 1-eps, 1+eps) * adv)
      - kl_weight * KL(...)
Output: Optimized policy model
```

i–ii

Grup kadar çıktı üret, her birini ödül fonksiyonuyla puanla

iii

Ödülleri grup içinde normalleştir: avantaj

iv

Kırpılmış oranla güncelle, KL cezasını çıkar

group_size genelde 4–16 · prob_ratio = current_prob / reference_prob

Birçok alanda en ileri düzey performans

Matematik · AIME 2024	Matematik · MATH-500	Kodlama · Codeforces	Kodlama · LiveCodeBench
%79,8	**%97,3**	**2029**	**%65,9**
Genel bilgi · MMLU	Genel bilgi · GPQA Diamond	Dil · AlpacaEval 2.0	Dil · FRAMES
%90,8	**%71,5**	**%87,6**	**%82,5**

Milyon girdi token'ı başına **0,14 $** API fiyatı

1,5B–70B boyutlarına damıtma; 7B model bile AIME 2024'te %55,5

12.3 · GRPO'NUN SINIRLAMALARI

Güçlü ama bedava değil: beş zorluk

Üretim maliyeti

Her istem için 4–16 tamamlama üretmek hesaplamayı artırır.

Yığın boyutu kısıtı

Gruplar birlikte işlenir; etkin batch küçülür, eğitim yavaşlayabilir.

Ödül tasarımı

Kötü ödül, istenmeyen davranışa veya yanlış hedefe götürür.

Grup boyutu dengesi

Az örnek çeşitlilik vermez, çok örnek süreyi ve kaynağı artırır.

KL ayarı

Çok yüksekse öğrenemez, çok düşükse başlangıçtan fazla uzaklaşır.

Ana fikir: GRPO, bir modeli akıl yürütmeye eğitmenin **daha verimli ve daha kararlı** yolu.

BÖLÜM 3 · SAYFA 12.3B VE 12.4

GRPO'yu ileri düzeyde anlamak ve TRL'de uygulamak

Avantaj formülü, hedef fonksiyon, kırpma, KL ve bunların kodu

3

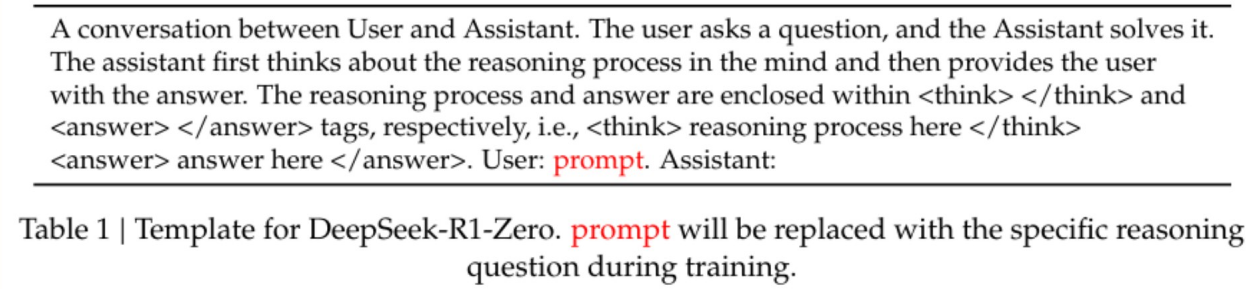
Eleştirmen yok: cevaplar kendi aralarında karşılaştırılır

Ayrı değer modeli (Critic) eğitilmez

Model aynı grupta ürettiği cevapları birbiriyle karşılaştırır. Hesaplama maliyeti önemli ölçüde azalır.

Doğrulanabilir her görevde çalışır

Örneğin matematikte cevap, gerçek cevapla (ground truth) karşılaştırılarak kolayca kontrol edilir.



A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within <think> </think> and <answer> </answer> tags, respectively, i.e., <think> reasoning process here </think> <answer> answer here </answer>. User: prompt. Assistant:

Table 1 | Template for DeepSeek-R1-Zero. prompt will be replaced with the specific reasoning question during training.

Görsel: DeepSeek-R1-Zero şablonu. Asistan önce düşünür; düşünceyi <think>, cevabı <answer> etiketlerine yazar.

Üç adım, tek örnek: "2 + 2 × 6 kaç eder?"

ADIM 1

Grup örnekleme

Her soru q için G = 8 cevap üret

ADIM 2

Avantaj hesaplama

Ödül ver, grup içinde standartlaştır

ADIM 3

Politika güncelleme

İyi cevapların olasılığını artır

$$\{o_1, o_2, \ldots, o_G\} \sim \pi_{\theta_{old}}(\cdot \mid q), \quad G = 8$$

o_1: 14 doğru | o_2: 16 yanlış | o_3: 10 yanlış | ... | o_8: 14 doğru

Bazı cevaplar doğru, bazıları yanlış. Bu **çeşitlilik** bir sonraki adım için çok önemli.

Bir cevap, grubun ortalamasından ne kadar iyi?

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \ldots, r_G\})}{\text{std}(\{r_1, r_2, \ldots, r_G\})}$$

Ödül r_i: doğru cevap **1**, yanlış cevap **0**. mean = ortalama, std = standart sapma.

Mutlak kalite ölçüsüne gerek yok; aynı gruptaki cevaplar karşılaştırılır.

8 cevap: 4 doğru, 4 yanlış

Ölçüt	Değer
Grup ortalaması	0,5
Standart sapma	0,53
Avantaj (doğru)	(1 – 0,5) / 0,53 = +0,94
Avantaj (yanlış)	(0 – 0,5) / 0,53 = −0,94

Kodla doğruladık: torch.std() bu değeri verir.

8 cevap, 4 doğru: avantaj dört adımda bulunur

1 **Ödülleri yaz**

```
r = [1, 1, 1, 1, 0, 0, 0, 0]
```

2 **Ortalamayı al**

```
mean = 4 / 8 = 0,5
```

3 **Standart sapmayı bul**

```
her farkın karesi (±0,5)² = 0,25 → toplam 2,0
2,0 / (8 - 1) = 0,286 → √0,286 = 0,53
```

4 **Farkı sapmaya böl**

```
doğru: (1 - 0,5) / 0,53 = +0,94
yanlış: (0 - 0,5) / 0,53 = -0,94
```

FORMÜLÜ CÜMLEYLE OKUYALIM

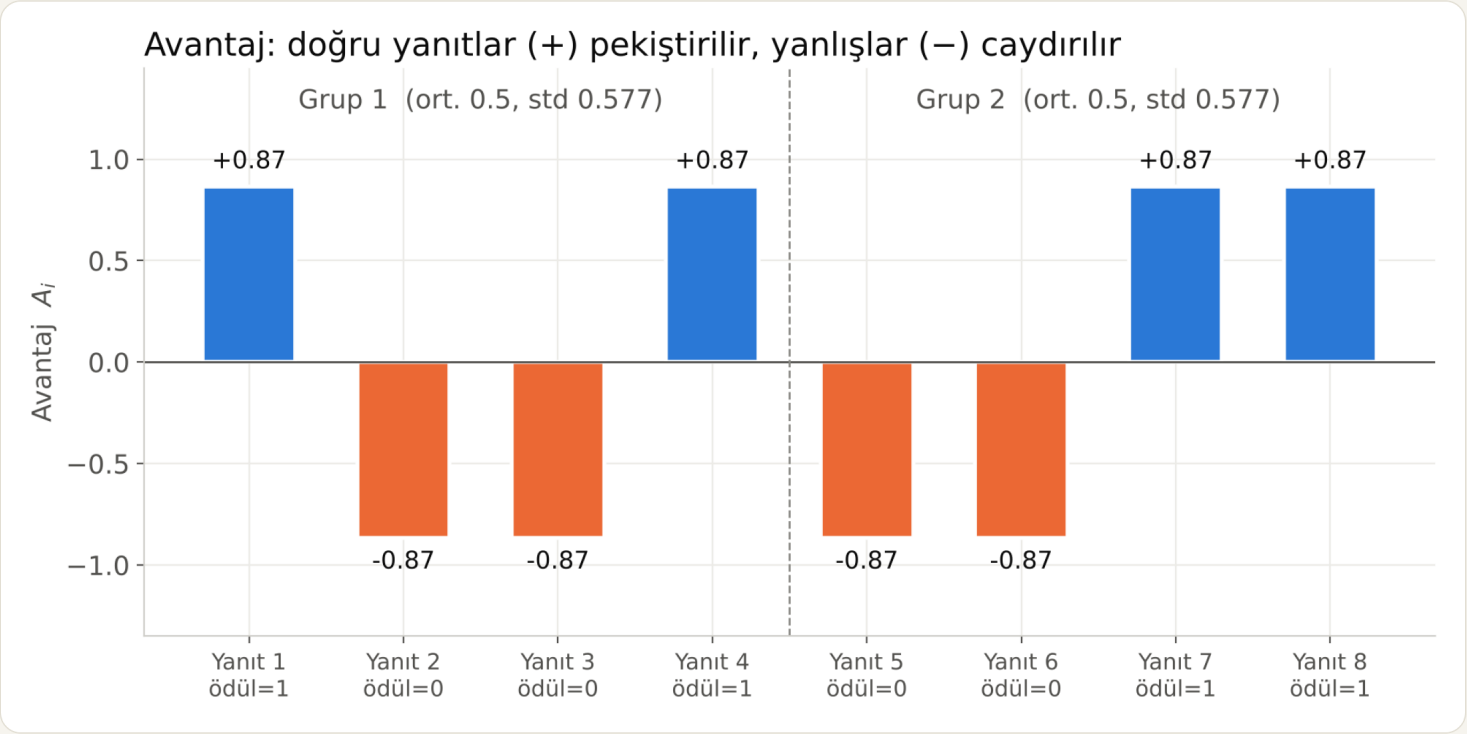
Bu cevap, grubun ortalamasından **ne kadar iyi?** Farkı, grubun ne kadar dağınık olduğuna bölerek ölçeklendiririz.

> **Neden ortalamayı çıkarıyoruz?**
>
> Ortalamanın üstü artı, altı eksi çıksın diye.

> **Neden std'ye bölüyoruz?**
>
> Grup içi normalleştirme, ödül ölçeği sorunlarını önler.

n – 1'e bölme, torch.std() varsayılanıdır.

Ortalamanın üstü pekiştirilir, altı caydırılır



$A_i > 0$

Cevap grubun ortalamasından iyi. Olasılığı artırılır.

$A_i < 0$

Cevap ortalamanın altında. Caydırılır.

Grafik: kurstaki kodun (2 grup × 4 cevap) bizim çalıştırdığımız çıktısı.

Hedef fonksiyon: korkutucu görünür, üç parçadan oluşur

$$J_{GRPO}(\theta) = \left[\frac{1}{G}\sum_{i=1}^{G}\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i,\ \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\epsilon, 1+\epsilon\right)A_i\right)\right] - \beta D_{KL}(\pi_\theta \,\|\, \pi_{ref})$$

1 · OLASILIK ORANI

Yeni model bu cevaba eskisine göre ne kadar daha çok olasılık veriyor?

2 · KIRPMA (CLIP)

Oranı [1 − ε, 1 + ε] aralığında tutar; sert değişiklik olmaz.

3 · KL IRAKSAMASI

β ağırlığıyla modeli referans modele yakın tutar.

Kurstaki formülde min içindeki iki terim arasındaki virgül eksik; DeepSeekMath makalesine ve sayfanın sözde koduna göre ekledik.

Formüldeki her sembol ne anlama geliyor?

$$J_{GRPO}(\theta) = \left[\frac{1}{G}\sum_{i=1}^{G}\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i,\ \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\epsilon, 1+\epsilon\right)A_i\right)\right] - \beta D_{KL}(\pi_\theta \,\|\, \pi_{ref})$$

`J_GRPO(θ)`	Büyütmek istediğimiz hedef (puan)
`θ`	Modelin eğitilen parametreleri
`q`	Soru (istem)
`G`	Bir soru için üretilen cevap sayısı
`o_i`	Gruptaki i'nci cevap
`A_i`	i'nci cevabın avantajı
`π_θ`	Yeni, güncellenen politika (model)
`π_θ_old`	Cevapları üreten eski politika
`π_ref`	Referans model (başlangıçtaki hâli)
`ε`	Kırpma sınırı: 0,2 ise aralık 0,8–1,2
`β`	KL cezasının ağırlığı (örn. 0,04)
`D_KL`	Yeni model ile referans arasındaki fark

Cümleyle: Her cevap için "oran × avantaj" ile "kırpılmış oran × avantaj"tan küçüğünü al, G cevabın ortalamasını hesapla, β × KL cezasını çıkar. Eğitim bu değeri büyütmeye çalışır.

Oran ne kadar büyürse büyüsün, en fazla 1,2

$$\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}$$

oran > 1: yeni model cevaba daha çok olasılık veriyor

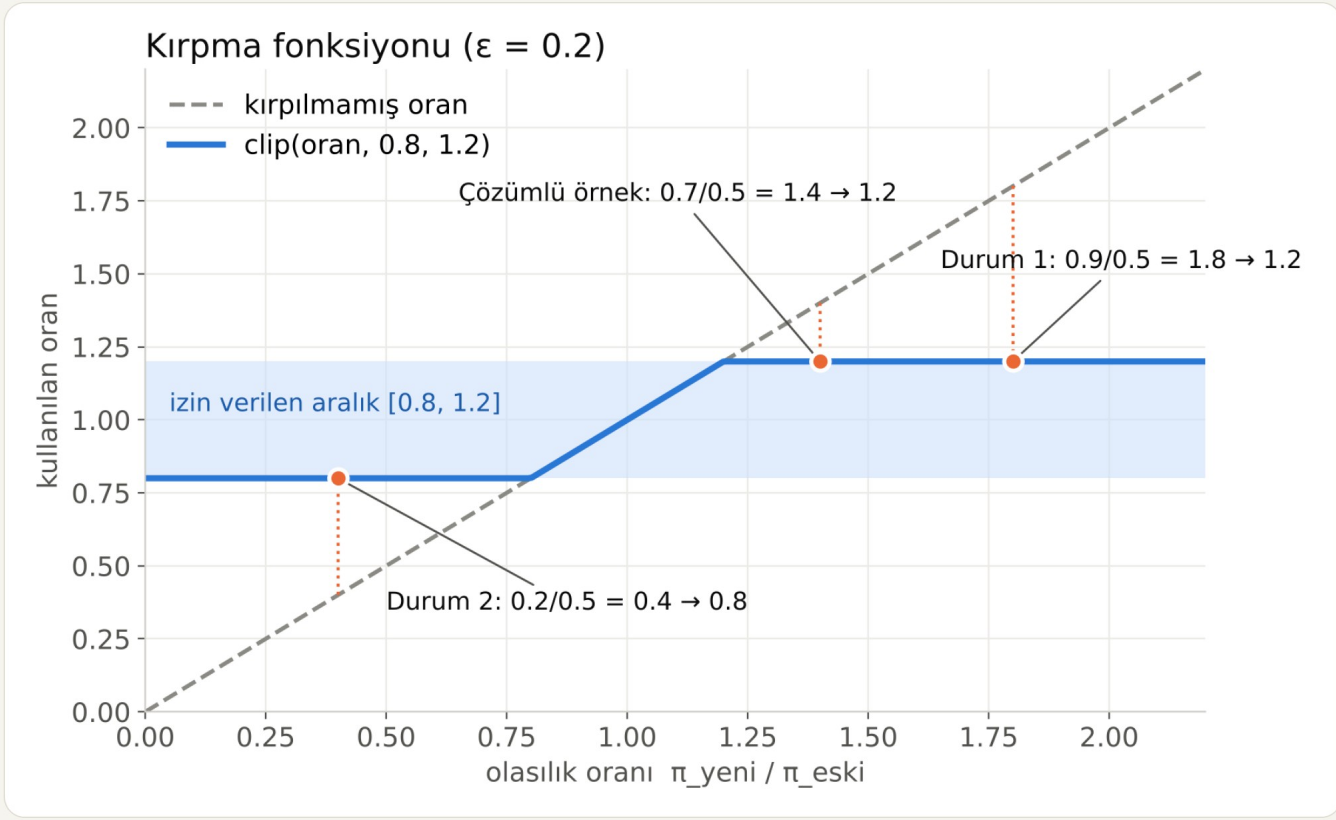
oran < 1: daha az olasılık veriyor

ε = 0,2 ile örnekler

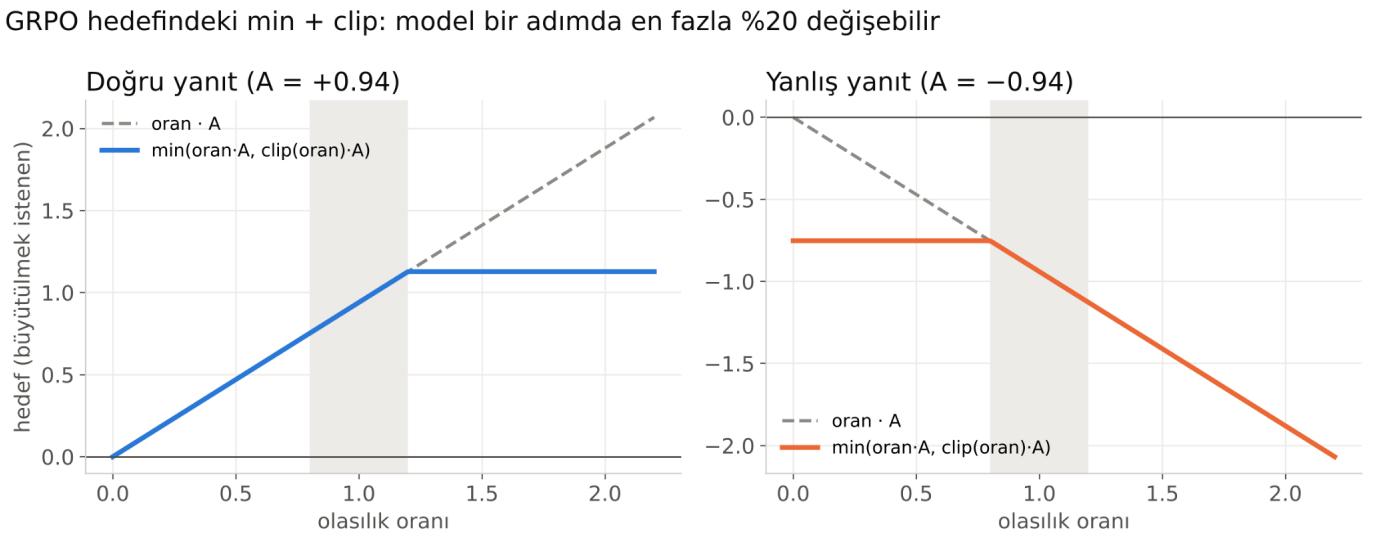
Durum 1: 0,9 / 0,5 = 1,8 → **1,2**

Durum 2: 0,2 / 0,5 = 0,4 → **0,8**

Çözümlü örnek: 0,7 / 0,5 = 1,4 → **1,2**



Güncelleme avantajla orantılı ama ılımlı



Eski modelin az olasılık verdiği cevap **sonucu iyileştiriyorsa** tercih edilir; zaten tercih edilen cevap ise yalnızca [0,8 – 1,2] içinde pekiştirilir.

Eski model kötü bir cevaba fazla değer verdiyse, yeni model bu yüksek olasılığı korumaktan **caydırılır**.

Güvenlik ağı: model orijinal davranışından fazla sapmasın

$$D_{KL}(P \| Q) = \sum_{x \in X} P(x) \log \frac{P(x)}{Q(x)}$$

P: yeni modelin dağılımı · Q: referans politikanın dağılımı

KL'yi küçük tutmak, modelin **anlamsız metin** veya aşırı yanlış cevap üretme riskini azaltır.

Keşfe (exploration) hâlâ izin verir.

Yüksek β

Model referansa yakın kalır; uyum yavaşlar, daha iyi cevapları keşfetmekte zorlanır.

Düşük β

Hızlı uyum ama kararsızlık; ödül istismarı (reward hacking) ve anlamsız çıktı riski.

DeepSeekMath

β = 0,04

12.3B · UYGULAMA ADIM 1: GRUP HÂLINDE CEVAP ÜRETMEK

Bir soruya 8 cevap üret, her birine ödül ver

```
model_name = "Qwen/Qwen2-Math-1.5B"
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

prompt = "Solve y = 2x + 1 for x = 2, y = " # doğru: 5
inputs = tokenizer(prompt, return_tensors="pt", padding=True)

batch_size, num_generations = 2, 4
outputs = model.generate(
    input_ids=inputs["input_ids"],
    attention_mask=inputs["attention_mask"],
    max_new_tokens=1, # cevap başına 1 token
    num_return_sequences=batch_size * num_generations,
    do_sample=True, top_k=10, temperature=0.7,
    pad_token_id=tokenizer.eos_token_id,
    return_dict_in_generate=True, output_scores=True,
)
```

ÇIKTI · 8 CEVAP

```
1: 5.0  2: 6.0  3: 7.0  4: 5.0
5: 10.0 6: 2.0  7: 5.0  8: 5.0
```

ÖDÜL · DOĞRU (5) = 1, YANLIŞ = 0

```
reward_1 = [1, 0, 0, 1]
reward_2 = [0, 0, 1, 1]
```

Ne yaptık? 2 grup × 4 cevap = 8 cevap. Her biri tek token. Doğru cevap 5 olduğu için 5 diyenler 1 aldı.

Kod kısaltıldı (cihaz ve model.eval satırları çevirideki tam hâlinde). Çıktı kurs sayfasından; modeli indiremediğimiz için bu adımı biz çalıştıramadık.

Kod ve bizim aldığımız çıktı

```
# 2 grup x 4 cevap, ödüller düzleştirilmiş
rewards = torch.tensor([1, 0, 0, 1, 0, 0, 1, 1],
    dtype=torch.float32)
num_generations = 4

rewards_grouped = rewards.view(-1, num_generations)
mean = rewards_grouped.mean(dim=1)
std = rewards_grouped.std(dim=1)

# her cevap için yay (broadcast)
mean = mean.repeat_interleave(num_generations)
std = std.repeat_interleave(num_generations)

advantages = (rewards - mean) / (std + 1e-8)
advantages = advantages.unsqueeze(1) # (8, 1)
```

ÇIKTI

```
Grouped Rewards:
[[1., 0., 0., 1.],
 [0., 0., 1., 1.]]
Mean per group: [0.5000, 0.5000]
Std per group:  [0.5774, 0.5774]

Advantages:
[ 0.8660, -0.8660, -0.8660,
 0.8660, -0.8660, -0.8660,
 0.8660,  0.8660]
Şekil: (8, 1)
```

Kurs sayfası 0.8659 gösteriyor; fark yalnızca yuvarlama. unsqueeze ile şekil logit'lere uyacak şekilde (8, 1) olur.

Oran, kırpma ve KL: formülün kod karşılığı

```
# 1) olasılık oranı (log farkının üssü)
ratio = torch.exp(new_per_token_logps
                  - per_token_logps)

# 2) kırpma, eps = 0.2
pg_losses1 = -advantages * ratio
pg_losses2 = -advantages * torch.clamp(
    ratio, 1.0 - eps, 1.0 + eps)
pg_loss_max = torch.max(pg_losses1, pg_losses2)

# 3) KL cezası, beta = 0.04
per_token_loss = pg_loss_max + beta * per_token_kl
```

ÇIKTI (örnek değerlerle)

```
ratio [1.4, 0.8, 0.4, 1.8]
clamp [1.2, 0.8, 0.8, 1.2]
loss1 [-1.212, 0.693, 0.346, -1.559]
loss2 [-1.039, 0.693, 0.693, -1.039]
max   [-1.039, 0.693, 0.693, -1.039]
İlk 4 cevap gösterildi · Şekil: (8, 1)
```

Eski olasılık 0,5; yeni olasılıklar kursun örneklerinden (0,7 · 0,9 · 0,2).

Neden max? Kayıp, hedefin eksi işaretlisi. Hedefte min almak, kayıpta max almakla aynı.

Bulgu: sayfadaki KL kodu bu şekilde hep 0 verdi. TRL'nin kodu KL'yi farklı bir tahminciyle hesaplıyor.

Sayfadaki KL kodu hep 0 veriyor: neden?

KURS SAYFASINDAKİ KOD

```
per_token_kl = F.kl_div(
    F.log_softmax(new_per_token_logps, dim=-1),
    F.softmax(per_token_logps, dim=-1),
    reduction="none",
).sum(dim=-1, keepdim=True)
```

```
per_token_kl : [0.0, 0.0, ... 0.0]
ortalama kayıp: -0.1298
```

Sebep: şekil (8, 1), son boyutta tek değer var. Tek değerin softmax'ı hep 1'dir; iki dağılım da aynı olduğu için KL = 0.

TRL'NİN KULLANDIĞI TAHMİNCİ

```
d = ref_per_token_logps - per_token_logps
per_token_kl = torch.exp(d) - d - 1
```

```
per_token_kl :
[0.051, 0.027, 0.584, 0.143,
 0.0, 0.006, 0.004, 0.016]
ortalama kayıp: -0.1257
```

Model referanstan ne kadar uzaklaşırsa ceza o kadar büyür; aynıysa 0.

Kaynak: TRL GRPOTrainer kaynak kodu (kursun 12.4'te bağlantı verdiği kütüphane). Değerler: bizim çalıştırmamız, örnek olasılıklarla.

12.3B · ÖZET

GRPO üç cümlede

1. Ayrı bir değer modeli olmadan, bir gruptaki **birden fazla çıktıyı karşılaştırır**.

2. Avantaj, ödülleri **standartlaştırarak** hangi cevabın ortalamanın üstünde veya altında olduğunu bulur.

3. Güncelleme, **kırpılmış hedef + KL cezası** ile kararlı öğrenme sağlar.

Doğruluğu nesnel olarak kontrol edilebilen matematik gibi görevlerde özellikle güçlü.

Dört parça yeter: veri, ödül, ayar, eğitici

```python
from trl import GRPOTrainer, GRPOConfig
from datasets import load_dataset
# 1. Veri kümesi
dataset = load_dataset("your_dataset", split="train")
# 2. Ödül fonksiyonu: uzun tamamlamayı ödüllendir
def reward_func(completions, **kwargs):
    return [float(len(completion)) for completion in completions]
# 3. Eğitim ayarları
training_args = GRPOConfig(
    output_dir="output", num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=2, logging_steps=10)
# 4. Başlat ve eğit
trainer = GRPOTrainer(
    model="your_model", # örn. "Qwen/Qwen2-0.5B-Instruct"
    args=training_args, train_dataset=dataset,
    reward_funcs=reward_func)
trainer.train()
```

NE YAPIYORUZ?

1 · Veri

Modele sorulacak istemler

2 · Ödül

Her tamamlamaya puan veren fonksiyon

3 · Ayar

GRPOConfig: süreci kontrol eden ayarlar

4 · Eğitici

GRPOTrainer hepsini birleştirir; grup oluşturma ve avantajı kendisi yapar

Cevap listesini alır, her cevap için bir sayı döndürür

ÖRNEK 1 · UZUNLUĞA DAYALI

```
def reward_length(completions, **kwargs):
    return [float(len(completion)) for completion in completions]
```

BİZİM ÇIKTIMIZ

```
"Short" → 5.0
"Medium length text" → 18.0
48 karakterlik cümle → 48.0
```

ÖRNEK 2 · DESENE (FORMATA) DAYALI

```
import re
def reward_format(completions, **kwargs):
    pattern = r"^<think>.*?</think><answer>.*?</answer>$"
    return [1.0 if re.match(pattern, c) else 0.0 for c in completions]
```

BİZİM ÇIKTIMIZ

Etiketler bitişik → **1.0**

Arada satır sonu var → **0.0**

Hiç etiket yok → **0.0**

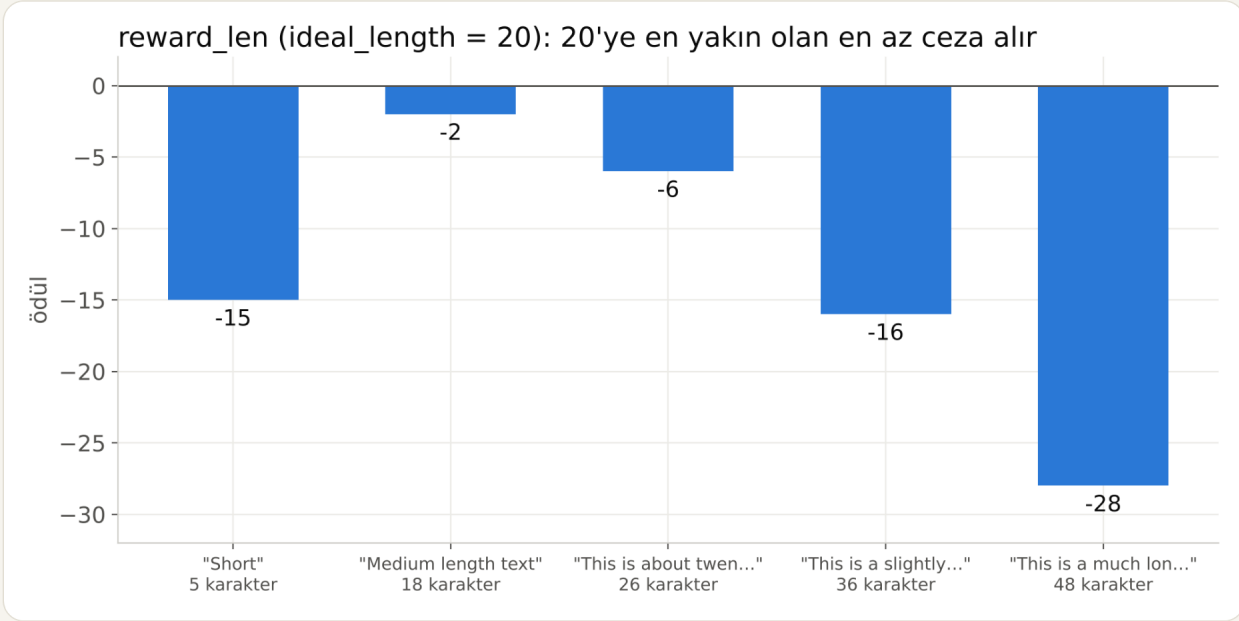
Kural: girdi = tamamlamaların listesi, çıktı = aynı uzunlukta sayı listesi. Desen katıysa küçük bir satır sonu bile ödülü sıfırlar.

Uzunluk tabanlı ödül: en kolay başlangıç

```python
def reward_len(completions, **kwargs):
    ideal_length = 20
    return [-abs(ideal_length - len(c))
            for c in completions]
```

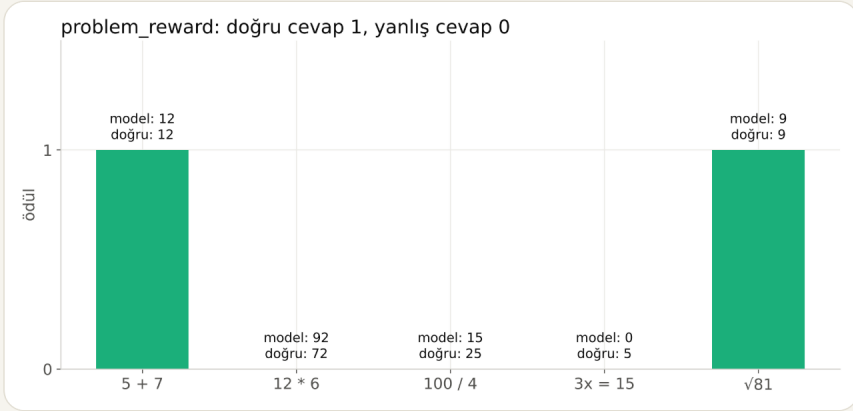
Çok kısa veya çok uzun cevaplar ceza alır. Ödül en fazla **0**; 20'ye ne kadar yakınsa o kadar iyi.

Örnek cümleler kursun gömülü not defterinden (grpo_length). len() karakter sayar.



Doğrulanabilir görevler: doğruysa 1, yanlışsa 0

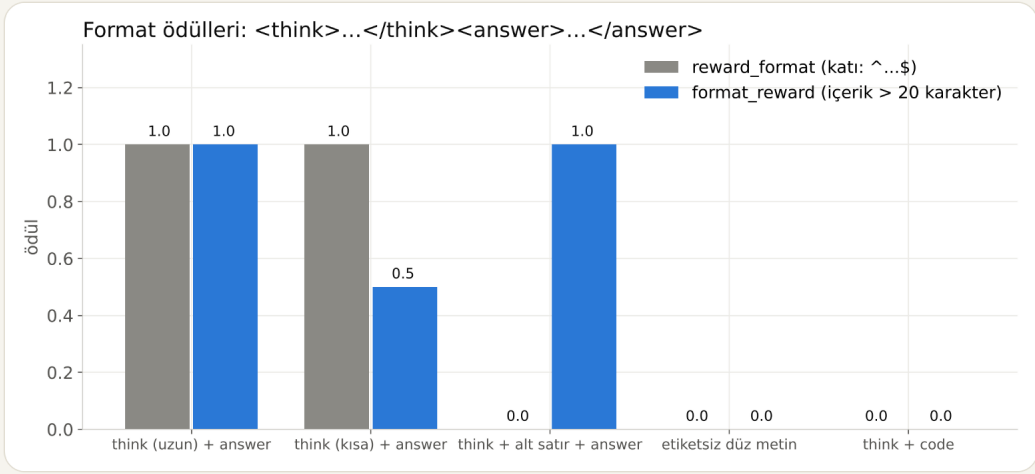
```python
def problem_reward(completions, answers, **kwargs):
    rewards = []
    for completion, correct_answer in zip(completions, answers):
        try:
            answer = extract_final_answer(completion)
            reward = 1.0 if answer == correct_answer else 0.0
            rewards.append(reward)
        except:
            rewards.append(0.0) # ayrıştırılamadı
    return rewards
```



Matematik ve kodlama gibi **nesnel olarak doğru cevabı olan** görevler için kural tabanlı ödül. extract_final_answer'ı görev için siz yazarsınız.

Format tabanlı ödül: önce düşün, sonra cevapla

```python
def format_reward(completions, **kwargs):
  pattern = (r"<think>(.*?)</think>\s*"
      r"<answer>(.*?)</answer>")
  rewards = []
  for completion in completions:
    match = re.search(pattern, completion, re.DOTALL)
    if match:
      think = match.group(1).strip()
      answer = match.group(2).strip()
      if len(think) > 20 and len(answer) > 0:
        rewards.append(1.0)
      else:
        rewards.append(0.5) # kısmi
    else:
      rewards.append(0.0)
  return rewards
```



Gri: sayfadaki katı örnek (^...$). Mavi: içerik kontrollü format_reward. Aynı cümlelerle bizim çalıştırmamız.

num_generations: grup boyutu, GRPO'nun kalbi

```
training_args = GRPOConfig(
  output_dir="output",
  num_train_epochs=3,
  num_generations=4, # istem başına
  per_device_train_batch_size=4,
  gradient_accumulation_steps=2,
  learning_rate=1e-5,
  use_vllm=True, # üretimi hızlandırır
)
```

ÇALIŞTIRINCA (TRL 1.16)

```
num_generation=4 → TypeError:
Did you mean 'num_generations'?
num_generations=4 → OK · beta=0.0 · epsilon=0.2
```

Grup boyutu nasıl seçilir?

2–3	Çok küçük: yeterli çeşitlilik yok
4–16	Önerilen: çeşitlilik ve maliyet dengesi
Daha çok	Öğrenme iyileşebilir, maliyet çok artar

İzlenecek metrikler

reward ortalama ödül · **reward_std** grup içi sapma · **kl** referanstan uzaklık

BÖLÜM 4 · SAYFA 12.5 VE 12.6

GRPO ile model eğitmek

TRL ile ince ayar, Unsloth ile GRPO ve kendi deneyimizin sonuçları

4

BÖLÜM 4 · UYGULAMANIN YOL HARITASI

İki alıştırma da aynı altı adımı izliyor

Adım	Ne yapıyoruz?	12.5 · TRL	12.6 · Unsloth
1 · Yükle	Veri kümesini ve modeli yükle	smoltldr · SmolLM 135M	GSM8K · Gemma 3 1B (4-bit)
2 · LoRA ekle	Tüm model yerine küçük ek katmanları eğit	r = 16	r = 32
3 · Ödülü yaz	İstenen davranışa puan veren fonksiyon	Uzunluk: 50'ye yakınlık	5 fonksiyon: doğruluk + format
4 · Ayarla	GRPOConfig: grup boyutu, öğrenme oranı, uzunluklar	num_generations = 8	num_generations = 6
5 · Eğit	GRPOTrainer(...).train() ve grafikleri izle	1 epoch · A10G ~1 saat	250 adım · Colab T4
6 · Dene, kaydet	Modeli yeni bir girdiyle dene ve paylaş	Hub'a yükle · pipeline	"Calculate pi." · LoRA / 16-bit / GGUF

Sonraki slaytların sol üstünde hangi adımda olduğumuz yazıyor. Kendi deneyimiz 12.5'i temel aldı.

Amaç: modele yaklaşık 50 token'lık cevap yazdırmak

Küçük bir dil modelini GRPO ile, **basit bir uzunluk ödülü** kullanarak eğitiyoruz.

Neden küçük model? Eğitim **sınırlı donanımda** da yapılabilsin. Güçlü donanımda 1.7B gibi büyük modeller denenebilir.

Model
SmolLM 135M Instruct

Veri kümesi
mlabonne/smoltldr

İnce ayar
LoRA (rank 16): küçük ek katmanlar, az bellek

Kütüphaneler
TRL (GRPOTrainer), PEFT, Datasets

Modelin tamamını değil, küçük LoRA katmanlarını eğitiyoruz

```
# Adım 1: veri kümesi ve model
dataset = load_dataset("mlabonne/smoltldr")
model_id = "HuggingFaceTB/SmolLM-135M-Instruct"
model = AutoModelForCausalLM.from_pretrained(
    model_id, torch_dtype="auto", device_map="auto",
    attn_implementation="flash_attention_2",
)
tokenizer = AutoTokenizer.from_pretrained(model_id)

# Adım 2: LoRA
lora_config = LoraConfig(
    task_type="CAUSAL_LM", r=16, lora_alpha=32,
    target_modules="all-linear",
)
model = get_peft_model(model, lora_config)
print(model.print_trainable_parameters())
```

ÇIKTI (KURS SAYFASI)

```
Total trainable parameters: 135M
```

NE YAPIYORUZ?

Veri

smoltldr: kısa hikâyelerden oluşan bir liste.

Model

135 milyon parametreli küçük bir sohbet modeli.

LoRA

Eğitilecek parametre sayısını, dolayısıyla gereken belleği azaltır.

Hedef 50: yaklaştıkça ödül artar

```
ideal_length = 50

def reward_len(completions, **kwargs):
    return [-abs(ideal_length - len(completion))
            for completion in completions]
```

Ödül = −| 50 − cevap uzunluğu |

50 → **0** (en yüksek) | 60 → **−10** | 96 → **−46**

GRPOConfig + GRPOTrainer = eğitim

```
training_args = GRPOConfig(
    output_dir="GRPO",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    gradient_accumulation_steps=2,
    max_prompt_length=512,
    max_completion_length=96,
    num_generations=8,  # her girdi için cevap sayısı
    num_train_epochs=1,
    bf16=True,
)
trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_len],
    args=training_args,
    train_dataset=dataset["train"],
)
trainer.train()
```

Ayarlar, transformers'taki **TrainingArguments** gibi GRPOConfig ile tanımlanır.

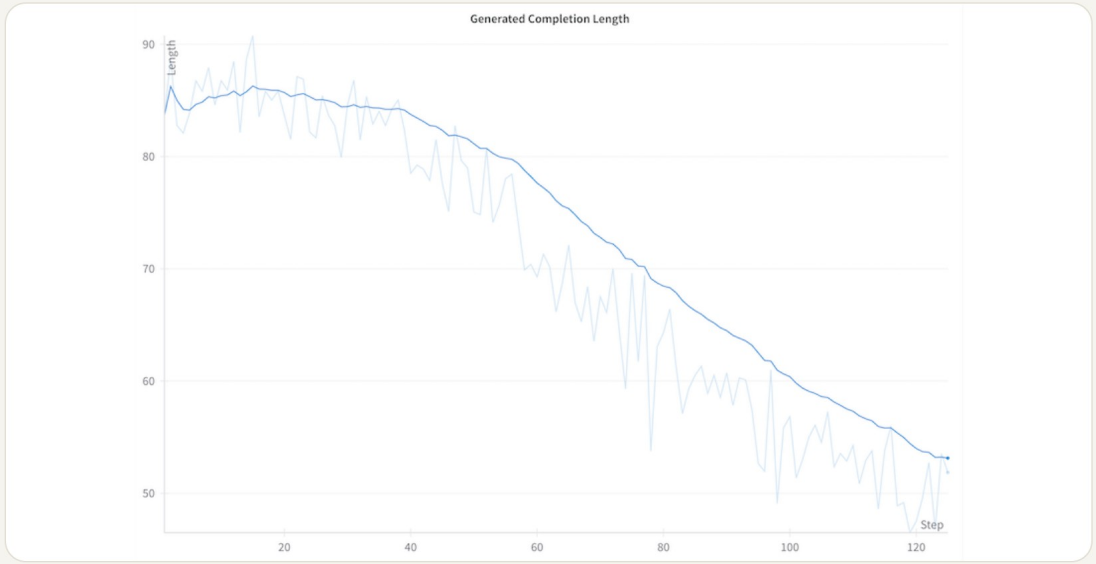
GRPOTrainer; model, ödül fonksiyonu, ayarlar ve veri kümesiyle başlatılır.

≈ 1 saat

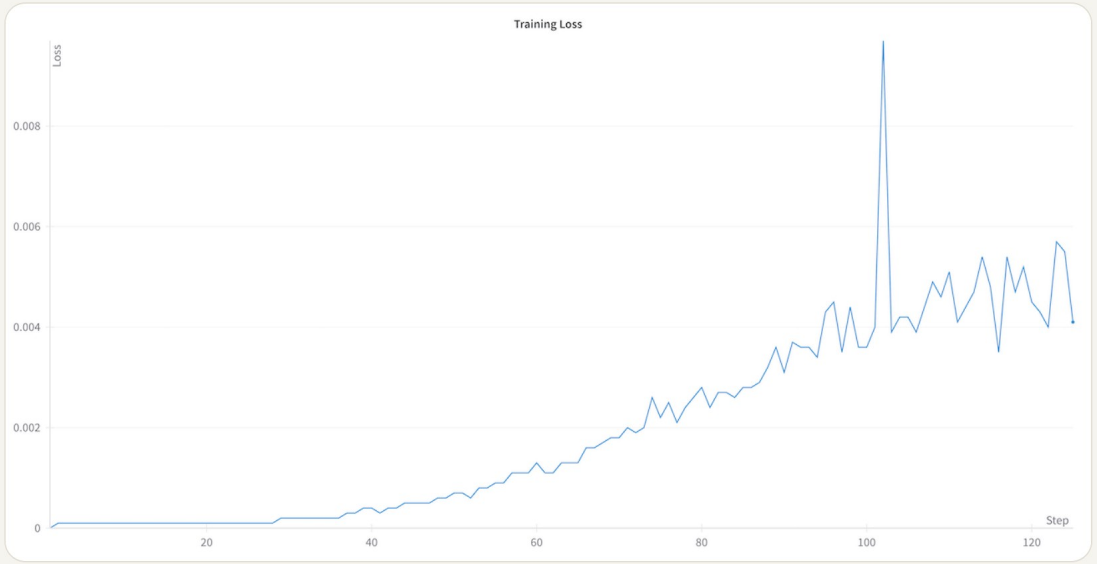
tek bir A10G GPU'da

Sonra LoRA katmanları ana modelle birleştirilip Hub'a yüklenebilir.

Kaybın artması burada iyi bir işaret



Cevap uzunluğu hedefe iniyor. Ödül 0'a yaklaşıyor; model doğru uzunluğu öğreniyor.



Kayıp 0'dan başlayıp artıyor. Beklenen durum: GRPO'da kayıp KL ıraksamasıyla orantılı.

LoRA'yı birleştir, Hub'a yükle, modele metin yazdır

```python
# LoRA katmanlarını ana modelle birleştir ve paylaş
merged_model = trainer.model.merge_and_unload()
merged_model.push_to_hub(
    "SmolGRPO-135M", private=False,
    tags=["GRPO", "Reasoning-Course"])

# Eğitilmiş modelle metin üret
messages = [{"role": "user", "content": prompt}]
generator = pipeline("text-generation", model="SmolGRPO-135M")
generate_kwargs = {
    "max_new_tokens": 256, "do_sample": True,
    "temperature": 0.5, "min_p": 0.1,
}
generated_text = generator(messages, generate_kwargs=generate_kwargs)
print(generated_text)
```

NE YAPIYORUZ?

Birleştir

Eğitilen LoRA katmanları ana modele eklenir; tek bir model olur.

Paylaş

Model Hugging Face Hub'a yüklenir.

Dene

İstem: kediler hakkında uzun bir belge. Model ondan metin üretir.

Modeli 4-bit yükle, sadece LoRA'yı eğit

Gemma 3 1B Instruct · GSM8K · Google Colab T4

```python
from unsloth import FastLanguageModel

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="google/gemma-3-1b-it",
    max_seq_length=1024,
    load_in_4bit=True,    # 4-bit: bellek tasarrufu
    fast_inference=True, # vLLM ile hızlı üretim
    max_lora_rank=32,
)
model = FastLanguageModel.get_peft_model(
    model,
    r=32, # LoRA rank: büyük = daha iyi ama yavaş
    target_modules=["q_proj", "k_proj", "v_proj", ...],
    lora_alpha=32,
    use_gradient_checkpointing="unsloth",
)
```

Amaç: model cevap vermeden önce düşünme adımlarını yazmayı öğrensin.

Unsloth: ince ayarı hızlandıran, daha az kaynakla yapan kütüphane.

Bu sayede eğitim **ücretsiz bir Colab GPU'sunda** bile yapılabilir.

Önce <reasoning>, sonra <answer>

```
SYSTEM_PROMPT = """
Respond in the following format:
<reasoning>
...
</reasoning>
<answer>
...
</answer>
"""
```

```
def get_gsm8k_questions(split="train"):
  data = load_dataset("openai/gsm8k", "main")[split]
  return data.map(lambda x: {
    "prompt": [
      {"role": "system", "content": SYSTEM_PROMPT},
      {"role": "user", "content": x["question"]},
    ],
    "answer": extract_hash_answer(x["answer"]),
  })
```

Model düşünme adımlarını **<reasoning>** içine, cevabı **<answer>** içine yazar. Doğru cevap veri kümesinden ayrıştırılır ve ödül hesabı için saklanır.

Beş ödül birlikte: doğruluk + format

```
def correctness_reward_func(prompts, completions, answer, **kwargs):
    responses = [c[0]["content"] for c in completions]
    extracted = [extract_xml_answer(r) for r in responses]
    return [2.0 if r == a else 0.0 for r, a in zip(extracted, answer)]
```

Ödül fonksiyonu	Neyi ödüllendirir	Puan
correctness	Cevap doğruysa	2,0
int	Cevap bir sayıysa	0,5
strict_format	Format birebir uyuyorsa	0,5
soft_format	Format genel olarak uyuyorsa	0,5
xmlcount	XML etiketleri doğruysa (fazla içeriğe ceza)	en fazla 0,5

Sayı mı, format katı mı, esnek mi?

```python
def int_reward_func(completions, **kwargs):
    responses = [completion[0]["content"] for completion in completions]
    extracted_responses = [extract_xml_answer(r) for r in responses]
    return [0.5 if r.isdigit() else 0.0 for r in extracted_responses]

def strict_format_reward_func(completions, **kwargs):
    pattern = r"^<reasoning>\n.*?\n</reasoning>\n<answer>\n.*?\n</answer>\n$"
    ... # eşleşirse 0.5, yoksa 0.0

def soft_format_reward_func(completions, **kwargs):
    pattern = r"<reasoning>.*?</reasoning>\s*<answer>.*?</answer>"
    ... # eşleşirse 0.5, yoksa 0.0
```

int

<answer> içindeki cevap bir sayıysa 0,5.

strict_format

Satır sonları dahil birebir aynı düzen. Başı (^) ve sonu ($) sabit.

soft_format

Etiketler doğru sırada olsun yeter; aradaki boşluk serbest (\s*).

Her soruya 6 cevap, beş ödülle puanlama

```
training_args = GRPOConfig(
  learning_rate=5e-6,
  num_generations=6, # soru başına cevap
  max_prompt_length=256,
  max_completion_length=768,
  max_steps=250,
)
trainer = GRPOTrainer(
  model=model, processing_class=tokenizer,
  reward_funcs=[xmlcount_reward_func,
    soft_format_reward_func, strict_format_reward_func,
    int_reward_func, correctness_reward_func],
  args=training_args, train_dataset=dataset,
)
trainer.train()
```

Ödüllerin belirgin artması **150–200 adım** sürebilir.

Test: eğitimde görülmemiş **"Calculate pi."** sorusu. Önce düşünme, sonra cevap beklenir.

KAYDETME SEÇENEKLERİ

- LoRA ağırlıkları
- 16-bit birleştirilmiş model
- GGUF (llama.cpp için)

Eğitimde görülmemiş bir soru: "Calculate pi."

```
model.save_lora("grpo_saved_lora")

text = tokenizer.apply_chat_template([
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": "Calculate pi."},
], tokenize=False, add_generation_prompt=True)

sampling_params = SamplingParams(
    temperature=0.8, top_p=0.95, max_tokens=1024)
output = model.fast_generate(
    text, sampling_params=sampling_params,
    lora_request=model.load_lora("grpo_saved_lora"),
)[0].outputs[0].text

# 16-bit birleştirilmiş model olarak kaydet
model.save_pretrained_merged("model", tokenizer,
    save_method="merged_16bit")
```

NE YAPIYORUZ?

Kaydet

Eğitilen LoRA ağırlıkları ayrı bir klasöre yazılır.

Dene

Aynı sistem istemiyle yeni bir soru sorulur; LoRA yüklenerek cevap üretilir.

Beklenen

Önce <reasoning> içinde düşünme, sonra <answer> içinde cevap.

Hub'a yükleme ve GGUF kaydı için kurs push_to_hub_merged ve push_to_hub_gguf kullanır.

GEÇIŞ · KURSTAKI UYGULAMALAR VE BIZIM DENEYIMIZ

12.5'i temel aldık, iki şeyi değiştirdik

	Kurs 12.5	Kurs 12.6	Bizim deneyimiz
Ölçüt	**TRL + PEFT**	**Unsloth + TRL**	**TRL + PEFT**
Model	SmolLM 135M Instruct	Gemma 3 1B Instruct	SmolLM2-135M-Instruct
Veri kümesi	smoltldr	GSM8K (matematik)	smoltldr
Ödül	Uzunluk (karakter)	5 fonksiyon	Uzunluk (token)
Girdi başına cevap	8	6	8
Eğitim	1 epoch	250 adım	1 epoch (1.000 adım)
Donanım	A10G, ~1 saat	Colab T4	RTX 4050, ~5 saat
Değerlendirme	Eğitim grafikleri	"Calculate pi." testi	10 örnekte önce/sonra

DENEY · SÜREÇ

İlk deneme başarısız oldu, sorunun kaynağını gösterdi

İLK DENEME

256 eğitim örneği, 60 adım

37,79 → 40,33

Ortalama hata **kötüleşti**. Bazı girdiler token sınırında kesildi; model ilgisiz veya yarım cevaplar üretti.

DÜZENLENMİŞ DENEY

2.000 örnek, 1 epoch, 1.000 adım, ~5 saat (RTX 4050)

Hiçbir girdi kesilmedi

En uzun girdi 442 token (sınır 512). Değerlendirme: eğitimde kullanılmamış 10 örnek, önce ve sonra aynı örnekler, rastgelelik kapalı (greedy).

Token kelimeyle aynı şey değildir; kelime parçaları ve noktalama işaretleri de birer token olabilir.

DENEY · GENEL SONUÇLAR

Hedefe ortalama uzaklık 43,2'den 9,5 token'a indi

Ölçüm	Önce	Sonra
Ortalama cevap uzunluğu	86,4	55,5
50 token hedefine ortalama uzaklık	43,2	9,5
96 token sınırına ulaşan cevap	8/10	1/10

%78

ortalama hatadaki azalma

DENEY · ÖRNEK BAZLI KARŞILAŞTIRMA

10 örneğin 9'unda hata azaldı

Örnek	Önce (token)	Sonra (token)	Önce hata	Sonra hata
1	96	48	46	2
2 · değişmedi	96	96	46	46
3	80	55	30	5
4 · tam hedef	96	50	46	0
5 · tam hedef	16	50	34	0
6	96	43	46	7
7	96	51	46	1
8	96	39	46	11
9	96	65	46	15
10	96	58	46	8

9 / 10

örnekte hata azaldı

2

cevap tam 50 token (örnek 4 ve 5)

8 → 1

sınıra ulaşan cevap sayısı

Örnek 5: 16 → 50 token. Model yalnızca kısaltmayı değil, **hedefe yaklaşmayı** öğrendi.

DENEY · SONUÇ VE SINIRLILIKLAR

Model, ödül fonksiyonu ne öğretirse onu öğrenir

SONUÇ

GRPO, ödül fonksiyonuyla tanımlanan bir davranışı küçük bir modele kazandırdı; ortalama hata **%78** azaldı.

İlk denemedeki başarısızlık, GRPO eğitiminde **veri hazırlığının** önemini gösterdi.

SINIRLILIKLAR

Sonuçlar doğruluğu veya akıl yürütmeyi değil, yalnızca **uzunluk davranışını** gösterir.

Değerlendirme **10 örnekle** yapıldı; daha geniş bir test kümesiyle doğrulanmalı.

1 örnekte (Örnek 2) model hâlâ üretim sınırına ulaştı.

Kaynak kod: [repo linki] · Kaynak: Hugging Face LLM Course, Bölüm 12.5

DEĞERLENDIRME SORULARI

Sıra sizde

KOLAY · AÇIK UÇLU

Pekiştirmeli öğrenmede "ödül" nedir? Bir dil modelinin eğitiminde nasıl kullanılır?

ORTA · AÇIK UÇLU

GRPO'nun hedef fonksiyonundaki KL ıraksama cezasının görevi nedir?

ZOR SORU · ÇOKTAN SEÇMELI

Bir soru için üretilen 8 cevabın hepsi doğru ve hepsi aynı ödülü (1) aldı. Ne olur?

A Tüm cevapların avantajı yüksek olur, model onları güçlü şekilde pekiştirir

B Tüm cevapların avantajı sıfır olur, bu grup güncellemeye katkı sağlamaz

C KL cezası devreye girer, model referans modele geri döner

D Clip fonksiyonu olasılık oranını 1+ε değerine sabitler

İpucu: avantaj formülünü hatırlayın, ödül – grup ortalaması.

KAPANIŞ

Ünitenin özeti

RL

Model deneme yanılmayla, ödüle bakarak öğrenir.

DEEPSEEK-R1

Saf RL ile akıl yürütme ve "aha anı" ortaya çıktı.

GRPO

Grup içi avantaj + kırpma + KL; eleştirmen modeli yok.

UYGULAMA

TRL ve Unsloth ile birkaç satırda eğitim; deneyimizde hata %78 azaldı.

KAYNAKLAR

Hugging Face LLM Course, Bölüm 12 (Apache 2.0) · DeepSeek-R1 (arXiv:2501.12948) · DeepSeekMath (arXiv:2402.03300) · Nathan Lambert, RLHF Book · TRL GRPOTrainer

Teşekkürler